LINFO2145 — Cloud Computing

Lesson I: Virtualization



Pr. Etienne Rivière

etienne.riviere@uclouvain.be

Course objectives

- Understand the concept of machine virtualization and its implementation
- Introduce the IaaS service model and cloud operating systems
- Detail OS-level virtualization and containers, and elements of their implementation
- Distinguish the benefits and uses of both approaches to virtualization

➡ Note: there will be a first quiz on the content of this single lecture, instead of 2 lectures normally

Part I - Machine virtualization

Introduction

- Virtualization abstracts resources presented to users and applications from physical resources
- Allows **decoupled** management of
  - a very, very large set of **virtual** cloud services and
  - the large set of **physical** machines supporting them

☑ Cost-efficiency

☑ Flexibility (unified OS/app management)

☑ Isolation and dependability

☐ Performance

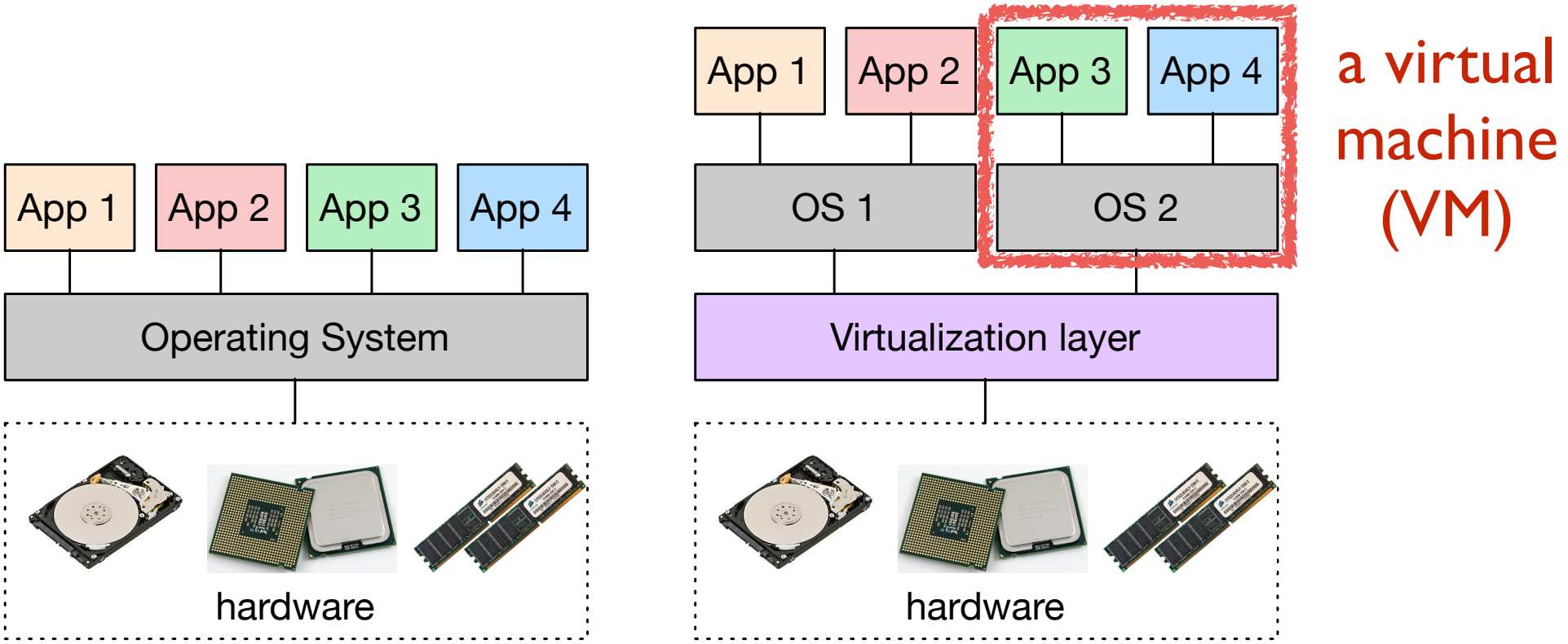
Basics: OS-level virtualization

- What we already know: an operating system (OS)

- One OS supports multiple applications, each running as a separate process
  - Cannot let each process access resources as it pleases
  - Yet, all processes must use memory, I/O, CPU time
  - ➡ *virtualization* of the resources by the OS
    - Virtual memory, file system, network primitives, etc.
  - ➡ with policies to arbitrate between competing demands

Machine virtualization

- One physical computer supports multiple *virtual* computers, sharing resources between several couples of OS and applications



non-virtualized system

virtualized system

A key enabling technology for Cloud computing

- 2014: more virtualized than non-virtualized servers [Koponen et al., NSDI 2014]

☑ Advantages for users
  - Resource abstraction and isolation
  - Deploy-on-demand: add & remove instances, migration
  - Enables the *pay-as-you-go* model

☑ Advantages for cloud operators
  - Reduces operational cost
  - Larger, more efficient servers supporting multiple VMs
  - Exploitation flexibility: migrate, consolidate, etc.

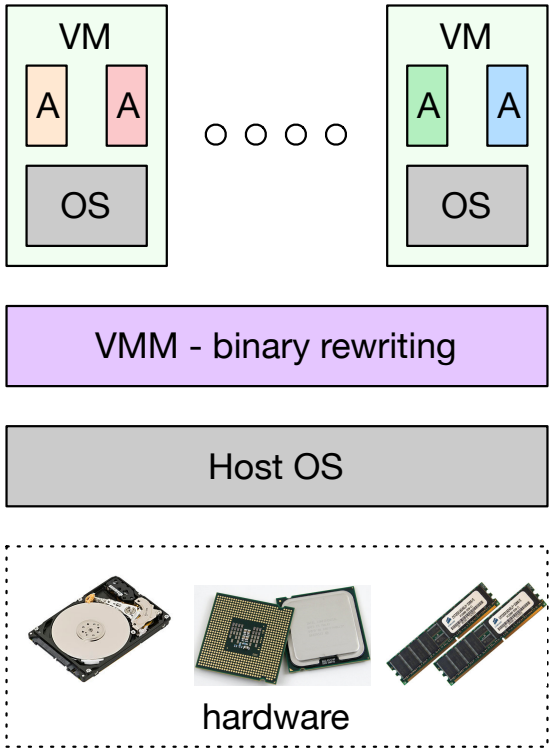
☐ Disadvantages
  - Costs of multiple OS state and maintenance
  - Performance penalty

Virtualization implementation

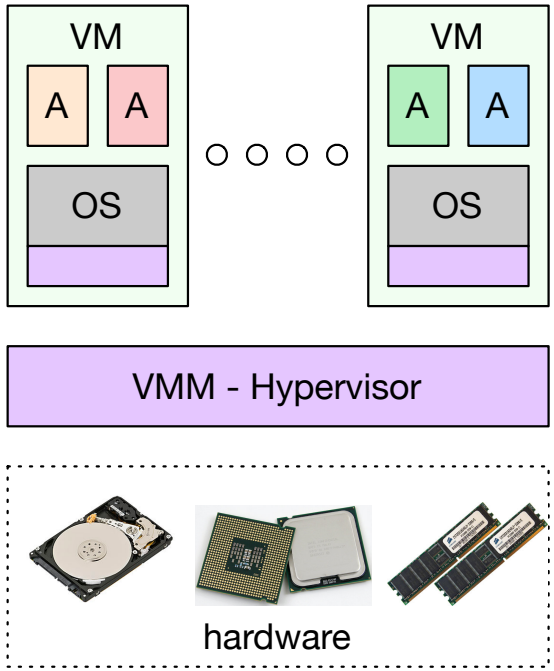
- Isolation in traditional OS relies on 2 instruction modes
  - 🐥 *Non-privileged*: can be used by user-space applications
    - e.g., accessing a virtual memory address not mapped to physical memory *traps* to the operating system for handling
  - 🐓 *Privileged*: only the kernel is allowed to use them

- Similar principle in a machine virtualization context
  - Non-privileged instructions execute normally
  - Privileged instructions
    - Some trap to the guest OS
    - Some trap to lower *virtualization* layer (even if used by the guest OS)
  - Implemented with more than 2 instruction modes (*aka* layers)
  - The hypervisor can emulate the behavior of unshared hardware

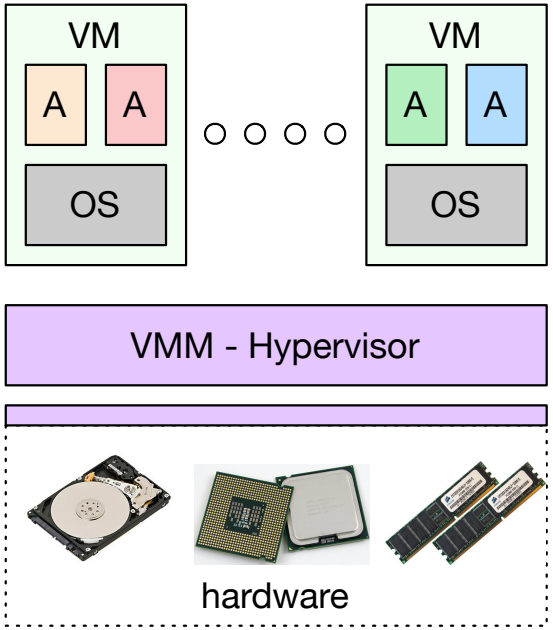
Server virtualization approaches



1st generation
Full virtualization by binary rewriting (software based)



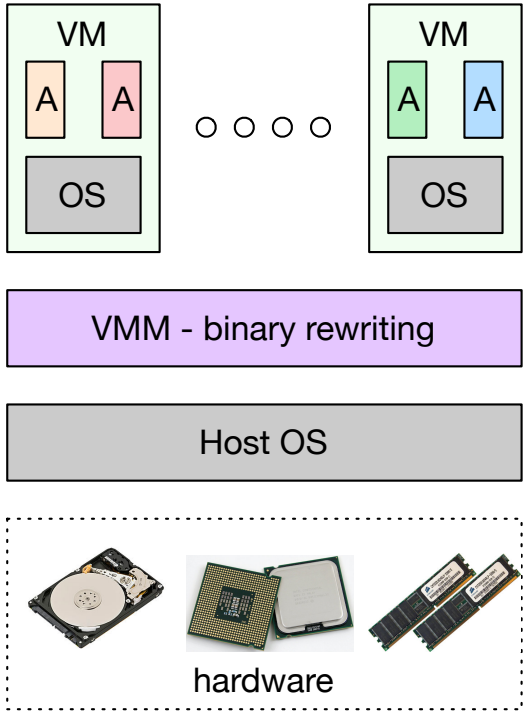
2nd generation
Paravirtualization
(software based collaborative virtualization)
Requires modified guest OS



3rd generation
Hardware-assisted full virtualization
(software+hardware based)

First-generation virtualization (full by binary rewriting)

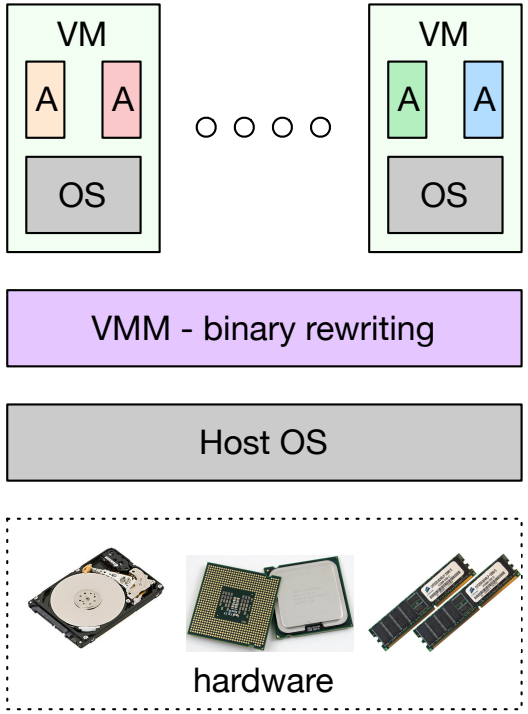
- Emulation layer in host operating system
- All the hardware is emulated (as virtual devices) including the CPU
  - Using binary rewriting: translating the instructions *on the fly*
- Guest OS not aware it is in an emulated environment
  - e.g., QEMU (http://wiki.qemu.org/)



1st generation
Full virtualization by binary rewriting (software based)

First-generation virtualization (full by binary rewriting)

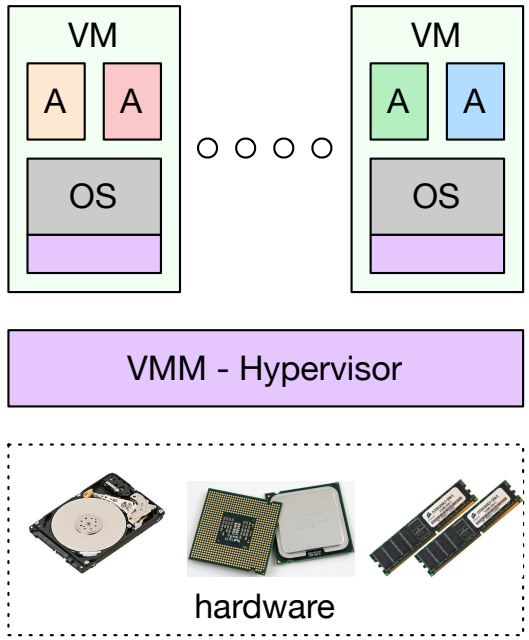
- Pros
  - ☑ Good VM isolation
  - ☑ Good control of VM resource usage
  - ☑ Total VM portability
    - Consistent (virtual) hardware across hosts
    - Emulate different architectures (very costly)
    - Useful for debugging and testing
- Cons
  - ☐ Huge performance penalty
  - ☐ Due to the privileged mode not being available to guest VMM, must *emulate* in software all privileged instructions



1st generation
Full virtualization by binary rewriting (software based)

Second-generation virtualization (paravirtualization)

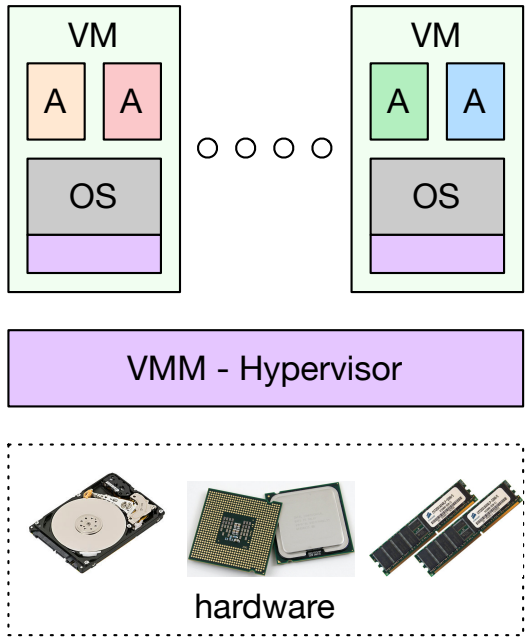
- Guest OS is modified
  - Does not need to run privileged instructions itself
  - Knows how to ask the Hypervisor to handle operations that require privileged instructions
    - "hypercall" interface, similar as OS system calls
  - Paravirtualized device drivers know how to efficiently access devices via hypervisor services
- Hypervisor arbitrates between guest OSes



2nd generation
Paravirtualization
(software based collaborative virtualization)
Requires modified guest OS

Second-generation virtualization (paravirtualization)

- Pros
  - ☑ Performance
  - ☑ OS awareness of virtualization allows inter-VM collaboration (e.g., for memory sharing or VM management)
- Cons
  - ☐ Requires OS vendor to provide paravirtualization-enabled version
    - Alternative: provide paravirtualization only partially through drivers (Microsoft Windows *enlightenments*, VirtualBox guest additions, etc.)



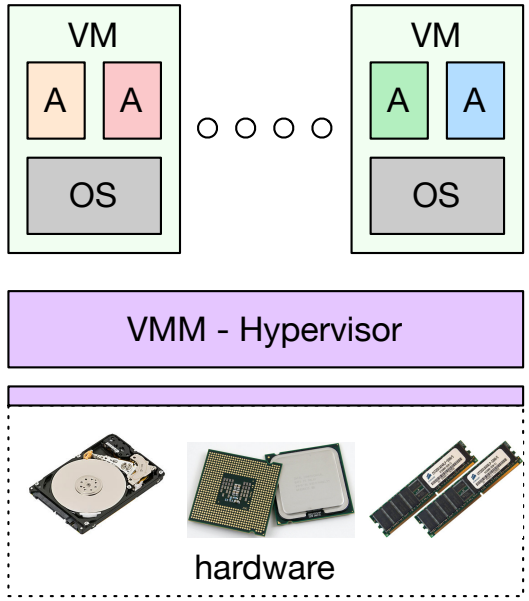
2nd generation
Paravirtualization
(software based collaborative virtualization)
Requires modified guest OS

Third-generation virtualization (full & hardware-assisted)

- Requires processor extensions
  - Intel-VT or AMD-V *(now everywhere)*
  - Can intercept and emulate performance-critical privileged operations in the guest OS in hardware
  - Hypervisor is still in charge of high-level management (e.g., resource allocation)

☑ Pro: can run an unmodified OS

☐ Cons: if the unmodified OS is not aware of virtualization, driver code may trigger lots of software traps, which can kill performance
  - still need paravirtualized device drivers for performance (we then talk about ***hybrid virtualization***)



3rd generation
Hardware-assisted full virtualization
(software+hardware based)

Virtualization and security

- Hypervisor-based *rootkits*
  - Could be installed on a non-virtualized system
  - Taking advantage of hardware-assisted virtualization
  - Successful *rootkit* can get higher privilege level than kernel and be almost impossible to detect
- Collocation attacks
  - Exploiting security flaws at the hypervisor level from one VM to inspect state of other VM
  - Exploiting data remaining in CPU cache after VM switch

⇒ Hypervisor security is critical

Desktop virtualization

- Run an guest OS inside the user host OS
  - Performance requirements less stringent
  - Integration with host OS required
  - Paravirtualization device drivers help for sharing devices (mouse, keyboard, network) and data more efficiently
- VMWare Fusion (commercial)
- VirtualBox (open source)
- Omnipath (open source, lightweight, cloud-native) — what you'll use in the project

VM images and operations

- Disk images
  - read/write: .VHD (most common), .VDI (Virtual Box), .QCOW2 (QEMU)
  - read-only: .iso (CD-ROM)
- VM image: disk image(s) + VM metadata (configuration, certificates)
  - most common: OVF/OVA Open Virtual Machine format
  - Amazon has its own format (AMI - Amazon Machine Image), imports OVA
- Migrate the VM from one host to another
  - Set up VM locally on Virtual Box, make an image,
  - deploy multiple instances on a IaaS private or public cloud
- Most VMM support suspend/resume of VMs
  - But generally not portable across VMMs

Part II - IaaS

Infrastructure-as-a-Service

- A service model allowing clients to rent infrastructure resources from the cloud
  - Virtual machines
  - Virtual networks
  - Storage (virtual disks)

- User in charge of installing and managing the OS and environment (libraries, configuration)

Key players in public cloud IaaS market







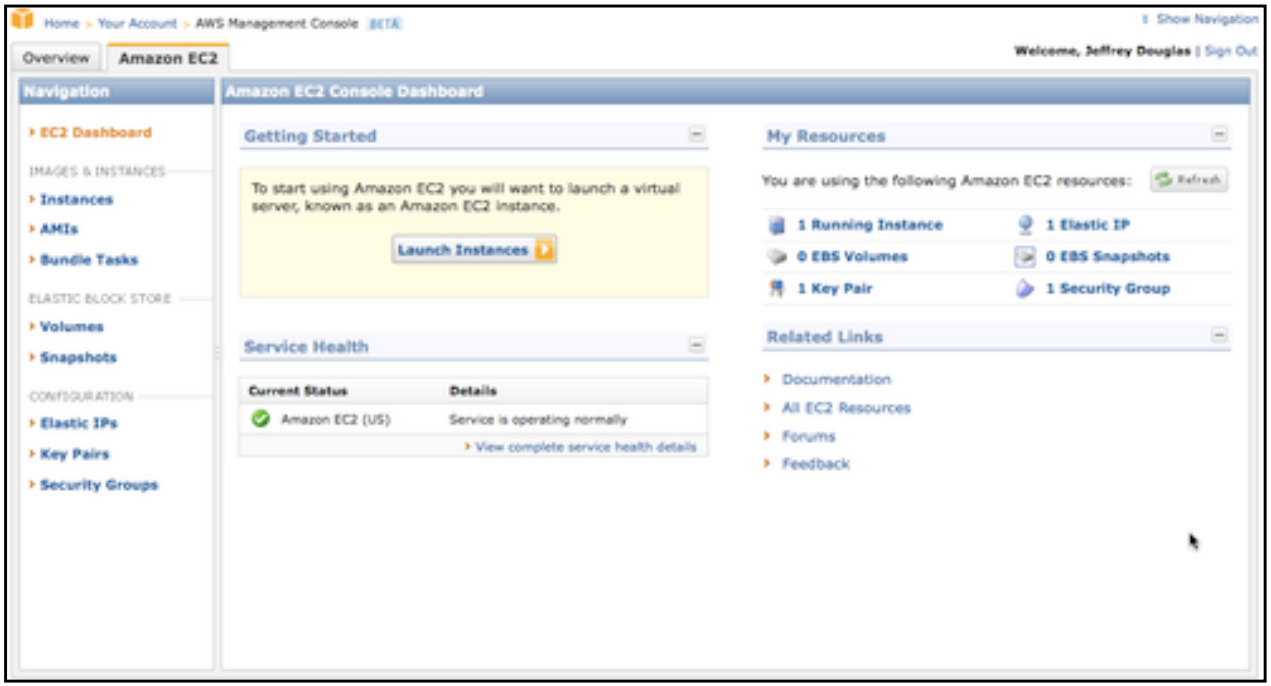






Public cloud IaaS operations

- Allocate a resource (VM, virtual network, etc.) on demand
- Deploy an image onto a host, and in some cases:
  - Migrate a VM between hosts
  - Change a VM configuration (amount of resources) dynamically
- Obtain configuration information (IP address)
- Manage access keys (SSH)



Defining needs: instances

- Users need to define the capacity of the VM they want to use
  - # virtual CPUs (VCPU) and memory
  - Type of storage (remote or dedicated on the host)

- Rackspace example (Linux instances)
  - General purpose
    - 1 to 8 VCPUS, 1 to 8 GB RAM, 20 to 160 GB SSD
  - Compute optimized
    - 2 to 32 VCPUS, 2 to 32 GB RAM, no SSD
  - I/O optimized
    - 4 to 32 VCPUs, 4 to 32 GB RAM, 40 GB system SSD + 150 to 1200 GB data SSD, high redundant bandwidth
  - Memory optimized
    - 2 to 32 VCPUs, 15 to 240 GB RAM, high redundant bandwidth

Typical IaaS pricing

Provider	Server (depending on instance type)	Storage	Network
Amazon EC 2	$0.113 to $6.82 / hour / VM	$0.095 to $0.125 / GB / month depending on location	IN: $0.01 per GB; OUT: first 1 GB free, then $0.09 down to $0.05 / GB decreasing with volume
Windows Azure	$0.02 to $1.60 / hour / VM	$0.07 to $0.12 / GB / month depending on redundancy	IN: free OUT: first 5 GB free, then $0.18 to $0.05 / GB decreasing with volume
Google Compute Engine	$0.019 to $1.66 / hour / VM ; charges by the minute (min 10 min.)	Virtual disk: $0.04 / GB / month; backup: $0.125 / GB / month	IN: free OUT: $0.23 to $0.08 / GB depending on volume and destination
Rackspace open Cloud	$0.04 to $5.44 / hour / VM	Files: $0.1 / GB / month Block: $0.12 / GB / month	IN: free OUT: BW; $0.11 down to $0.06 / GB, decreasing with volume

How is the fleet of hosts and resources managed?

- A data center is a very complex infrastructure
  - Specific hardware, network services, etc.
  - Lots of different clients and quick response time required

- Many operations common to all IaaS: allocating hosts, creating VMs, fault monitoring, usage monitoring and billing, etc.
  - ➡ IaaS management system
  - ➡ Open-source examples include OpenStack & OpenNebula
  - ➡ Commercial offerings, e.g., Nutanix

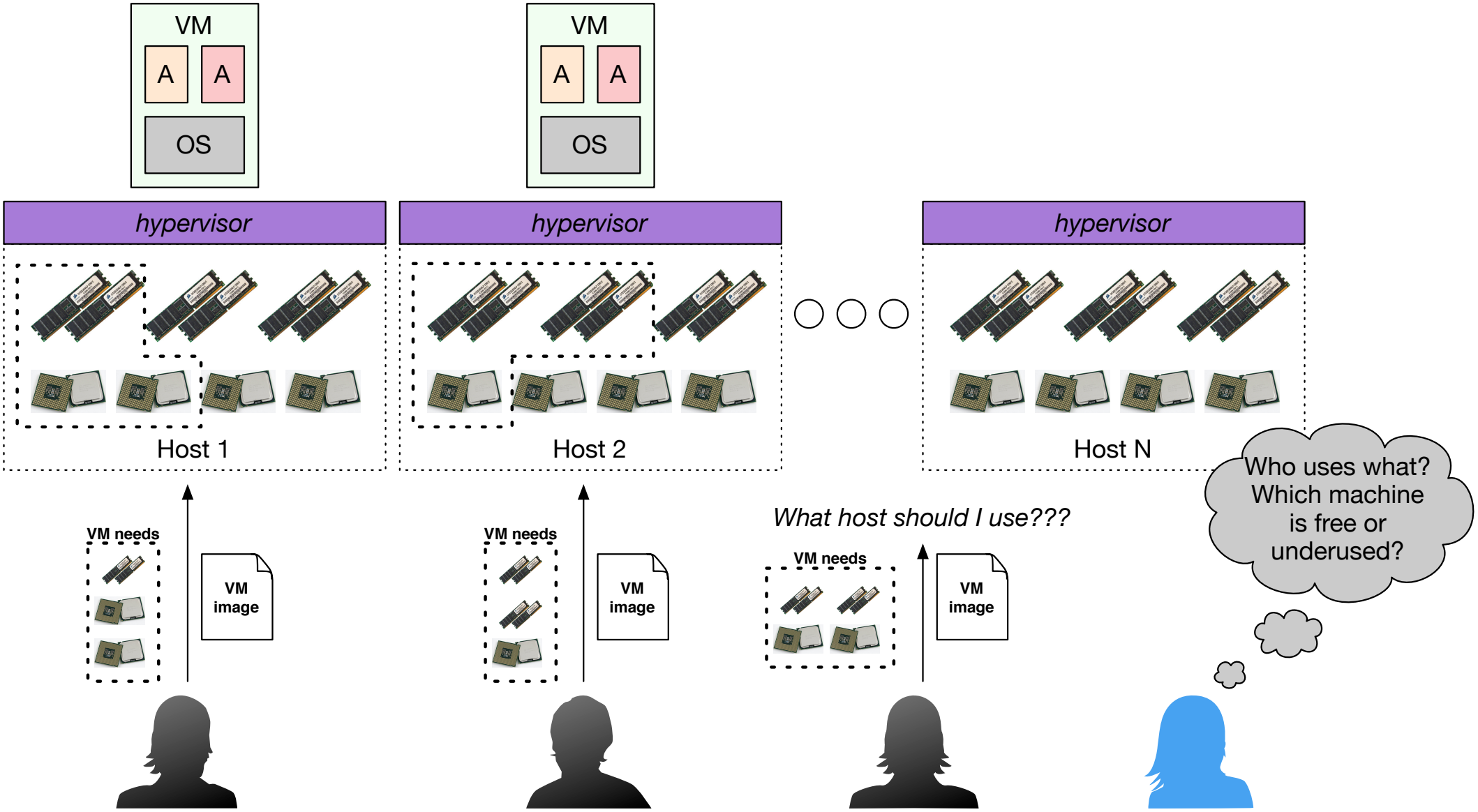




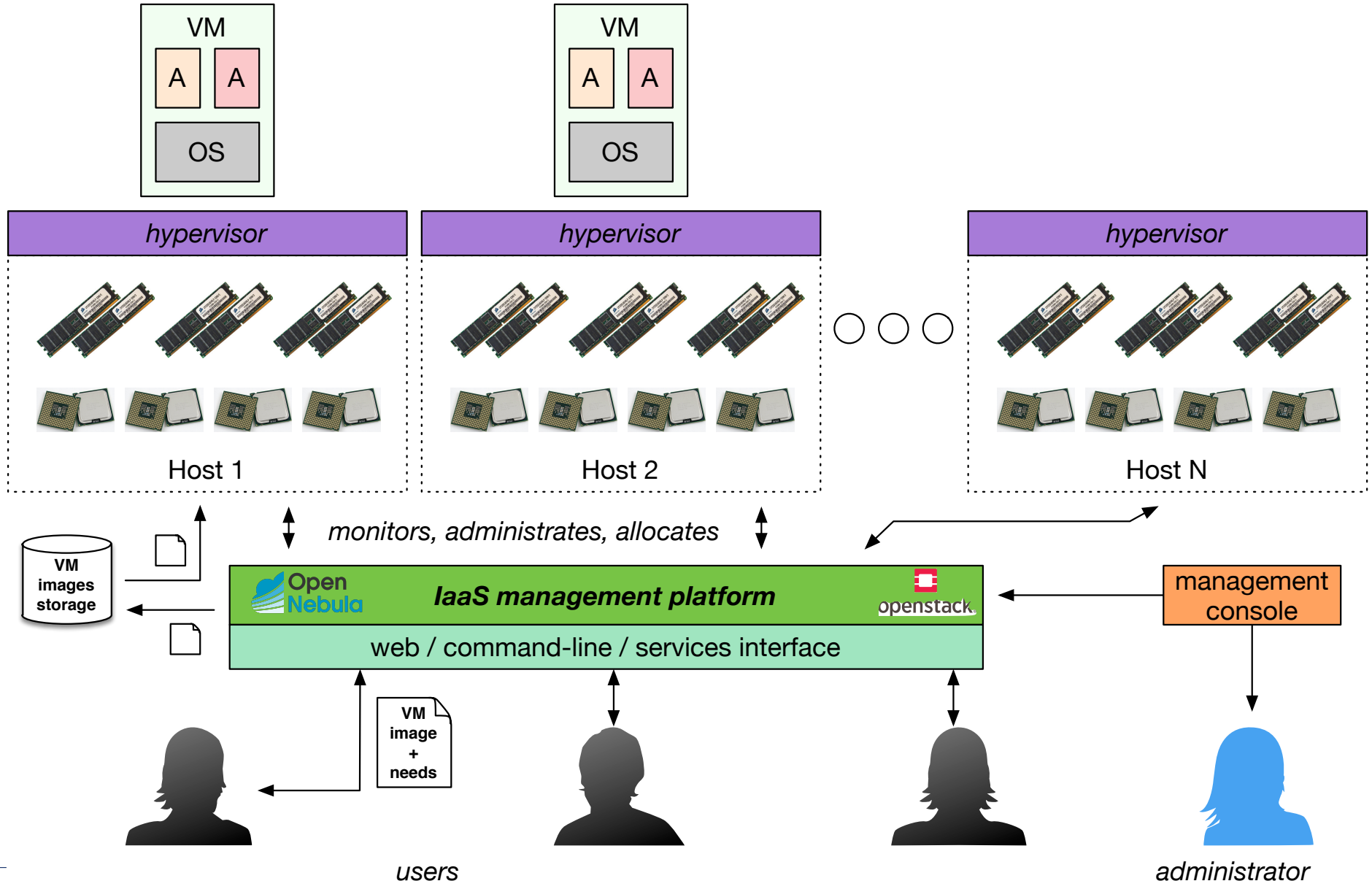
An operating system analogy

- IaaS management platforms can be considered as the "Cloud operating system" …
- … as their role is in many ways similar to the OS resource management role on a single machine
  - Run VMs instead of processes
  - Allocate (fixed) portions of RAM to different VMs instead of (dynamic) allocation using virtual memory
  - Inter-VM networking (with paravirtualization or hybrid virtualization, e.g. *virtio*) vs. IPC
  - Naming and configuration
  - Authentication, protection and security, etc.

Direct infrastructure access

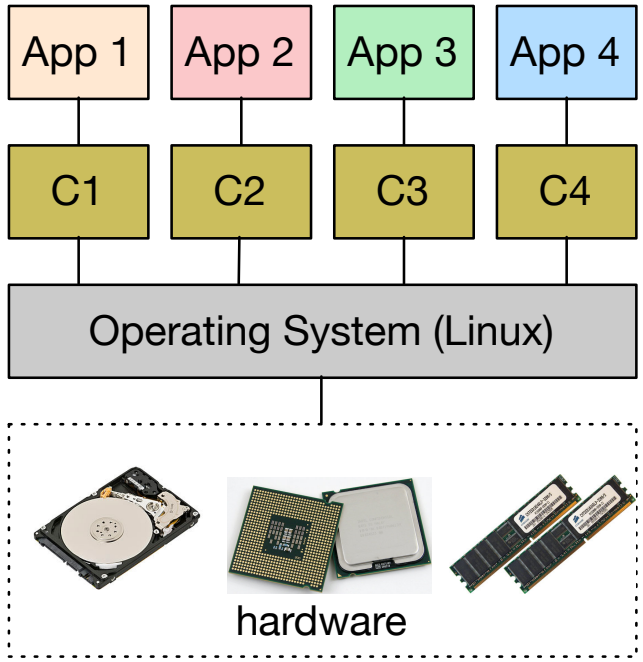


IaaS management platform



Part III - Containers

OS-level virtualization: basics



system supporting containers

- Instead of virtualizing a *machine*, we can virtualize the OS
  - Allows the independent management of *containerized* applications
- Machine virtualization decouples OS from hardware; containers decouple applications from OS
  - From a *Machine-oriented* to an *application-oriented* datacenter

Containers: requirements

- A single OS (kernel, device drivers, FS, etc.)
  - Multiple user-space instances over a single kernel-space instance
- Enabling *mechanism* = Isolation
  - No access or visibility across user-space instances
    - No sharing of process identifiers, file handlers, shared memory, file system (unless explicit), I/O devices, networking, …
  - ✓ Processes in one user-space instance believe that they have the machine just for themselves
  - ➡ Linux's **namespaces**
- Enabling *policy* = Resource management
  - Limit the use of resources by one container
  - Memory use, network use, disk quotas, etc.
  - ➡ Linux's control groups (**cgroups**)

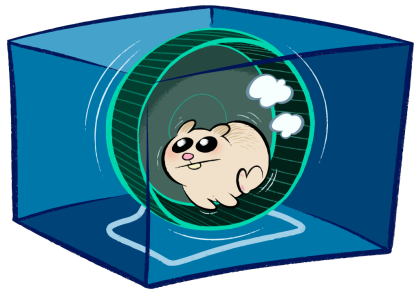
Linux containers

- Generic name for containers for Linux based on namespaces and cgroups
- Active open-source projects (by date of first availability)
  - OpenVZ (2005)
  - LXC (2008)
  - Linux-VServer (2013)
  - Open Container Initiative runC (2013)
  - Rocket (rkt) by CoreOS (2014)
- Generally considered as relatively complex to use, configure and automate
  - Docker simplifies these aspects









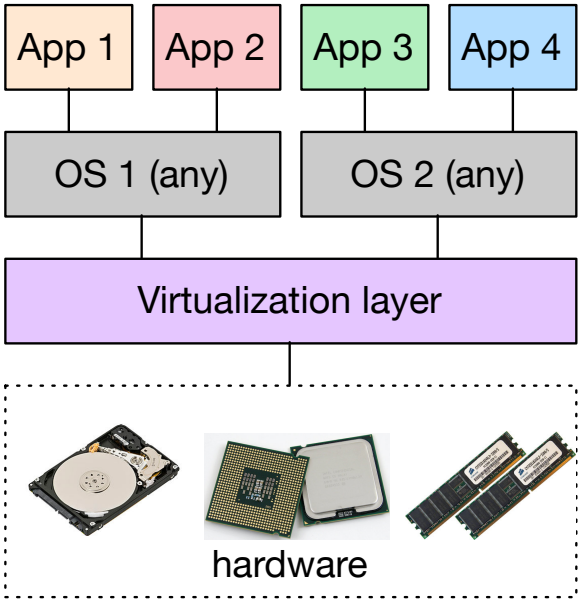


- Wrap-up complete applications inside containers
- The file system contains everything needed to run the application
  - Runtime, system tools, and system libraries
  - ✓ Solving the portability problem of earlier implementation of the container concept, which assumed the same complete OS distribution
- Offers full functional toolset including mechanisms for creating and manipulating portable containers
  - Even if OS different in dev/testing/prod
  - Linked to and used by the *DevOps* movement

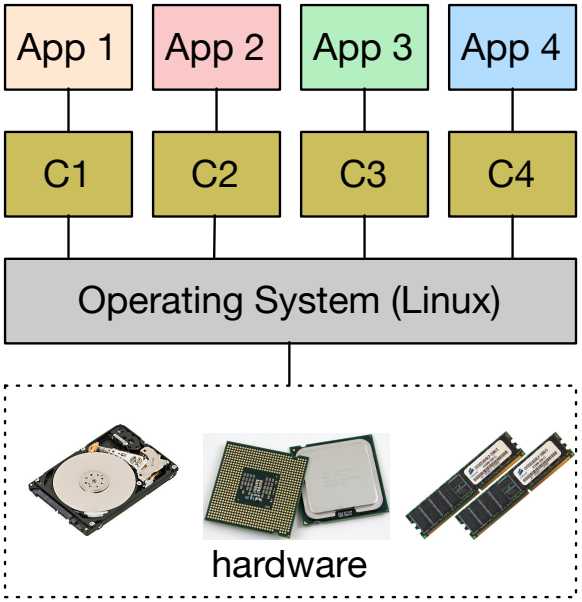
Containers vs virtual machines

- OS-level vs. machine virtualization
  - Boundary = system call interface vs. hardware emulation
- ✓ Less overheads
  - System calls in kernel modified to handle different "contexts"
  - Resource allocation and monitoring require little resources
- ✗ Less flexibility
  - No support for arbitrary environment, unless specific emulation in place (e.g. support of Windows applications in Docker, support of Solaris 8/9 applications in Solaris 10)
- ✗ Isolation is not as good as with Virtual Machines
  - Attack surface (Kernel system calls) larger than hypervisor attack surface
  - Some resources (level-3 caches in CPU) not isolated
- ➡ Best of both worlds: combine virtual machines and containers
  - ✓ Decouple hardware and OS management (cloud/tenant)
  - ✓ Decouple OS and applications management (applications/tenant)

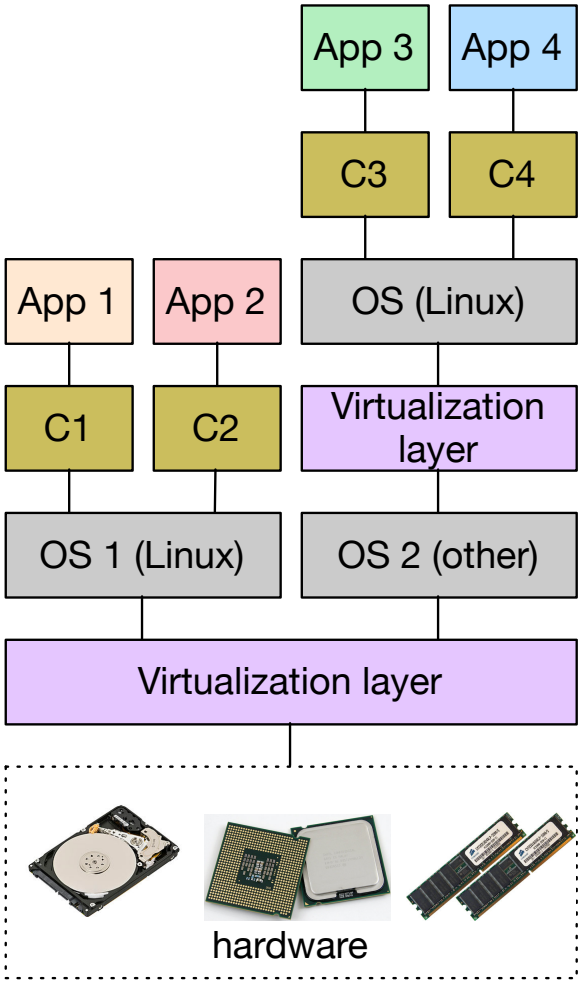
VMs and containers



virtualized system



system supporting containers



virtualized system supporting containers

Using Docker containers

Running a first container (1)

```
Etienne — root@6c04157c5f59: / — docker run -i -t ubuntu /bin/bash — 80×19
Last login: Wed Mar  1 17:06:00 on ttys004

<iMac-de-Riviere:etienne> 0 [03-01 17:06] ~ (0 Mb)
! docker run -i -t ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
d54efb8db41d: Pull complete
f8b845f45a87: Pull complete
e8db7bf7c39f: Pull complete
9654c40e9079: Pull complete
6d9ef359eaaa: Pull complete
Digest: sha256:dd7808d8792c9841d0b460122f1acf0a2dd1f56404f8d1e56298048885e45535
Status: Downloaded newer image for ubuntu:latest
root@6c04157c5f59:/# ls /
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@6c04157c5f59:/#
```

Pulling layered FS images from repository

this is the Ubuntu FS, no link to the host MacOS X

-i option keeps STDIN open from the container;
-t for creating a TTY allows running an interactive shell (here bash)

Running a first container (2)

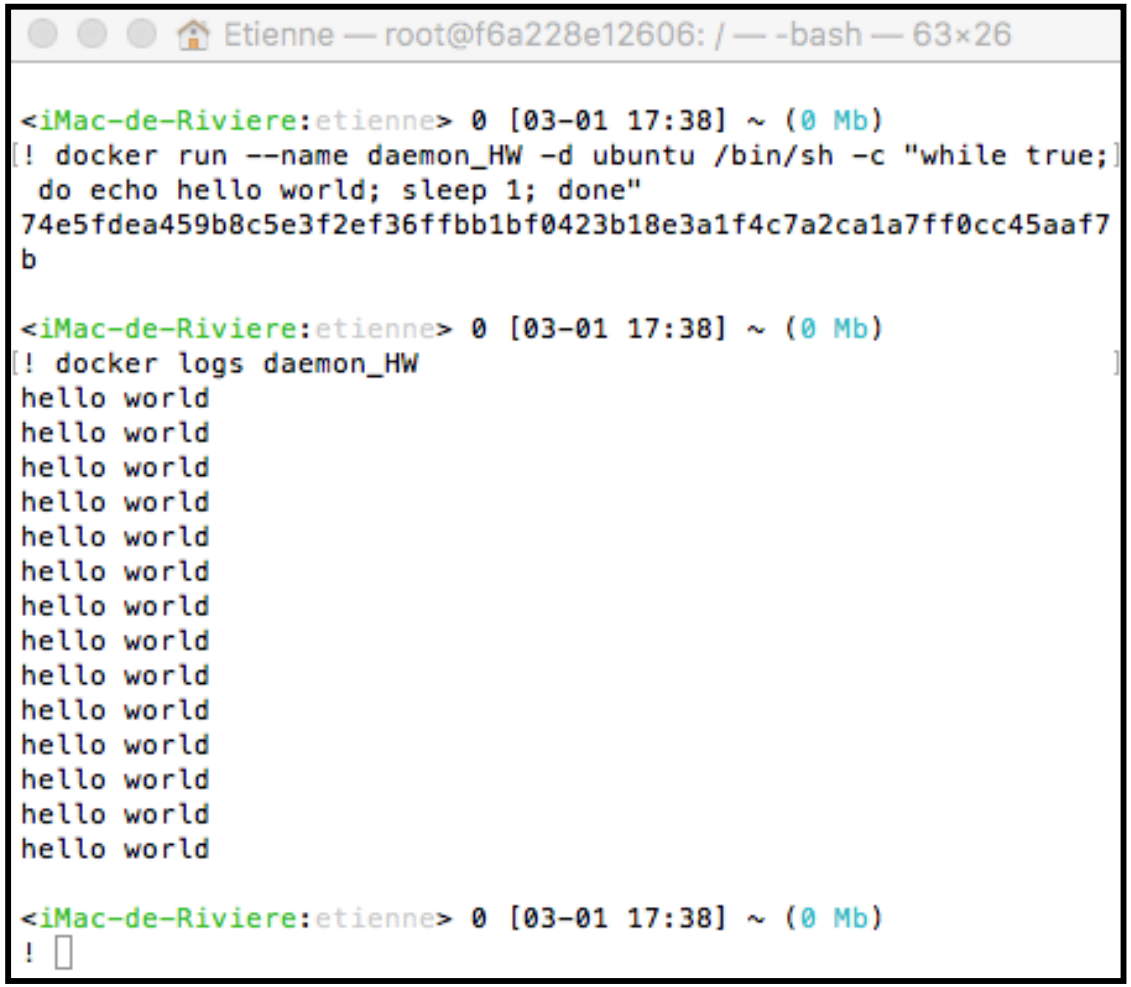
- `docker run -i -t ubuntu /bin/bash`
  - `docker`: docker client on the command line
  - `run`: command
  - `-i -t`: interactive shell with STDIN kept open
  - `ubuntu`: base image
    - by default, will expand to ubuntu:latest
    - can ask for a specific version, e.g., ubuntu:12.04
    - This image is provided by Docker inc. on the Docker Hub
  - `/bin/bash`: command to run after container creation
- Exit the container with the `exit` (or ctrl-D) command

Some common commands

- `docker run` (eq. to `docker create` + `docker start`)
- `docker ps`
  - List running containers
  - `-a`: all containers, running and stopped ones
  - `-l`: latest running container
- `docker start [identifier]`
  - identifier can be the UID (e.g., `f6a228e12606`), the automatically-generated name (e.g., `hardcore_rosalind`) or the name provided to `docker run` with the `--name` option
- `docker attach`
  - re-attach to an interactive session
- `docker stop | kill`
  - send `SIGSTOP` or `SIGKILL` to container
- `docker inspect [identifier]`
  - get details about a running container

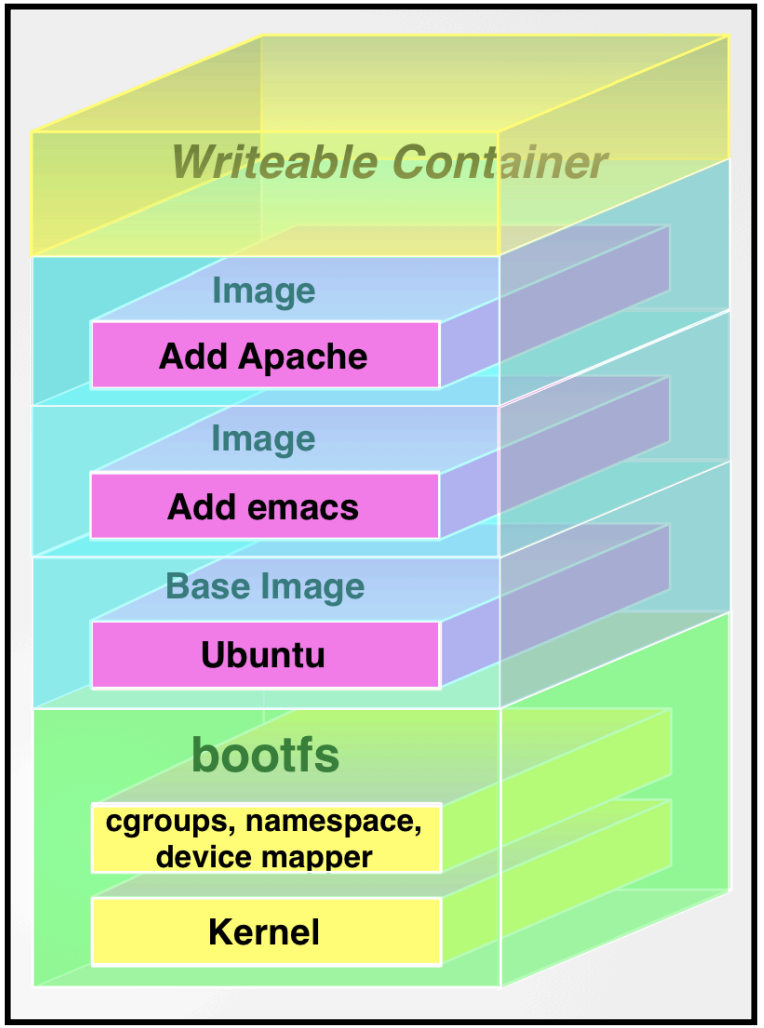
Daemonized containers

- Most common containers: no interactive shell
  - -d flag
- Can retrieve logs with `docker logs`



Content of a docker image

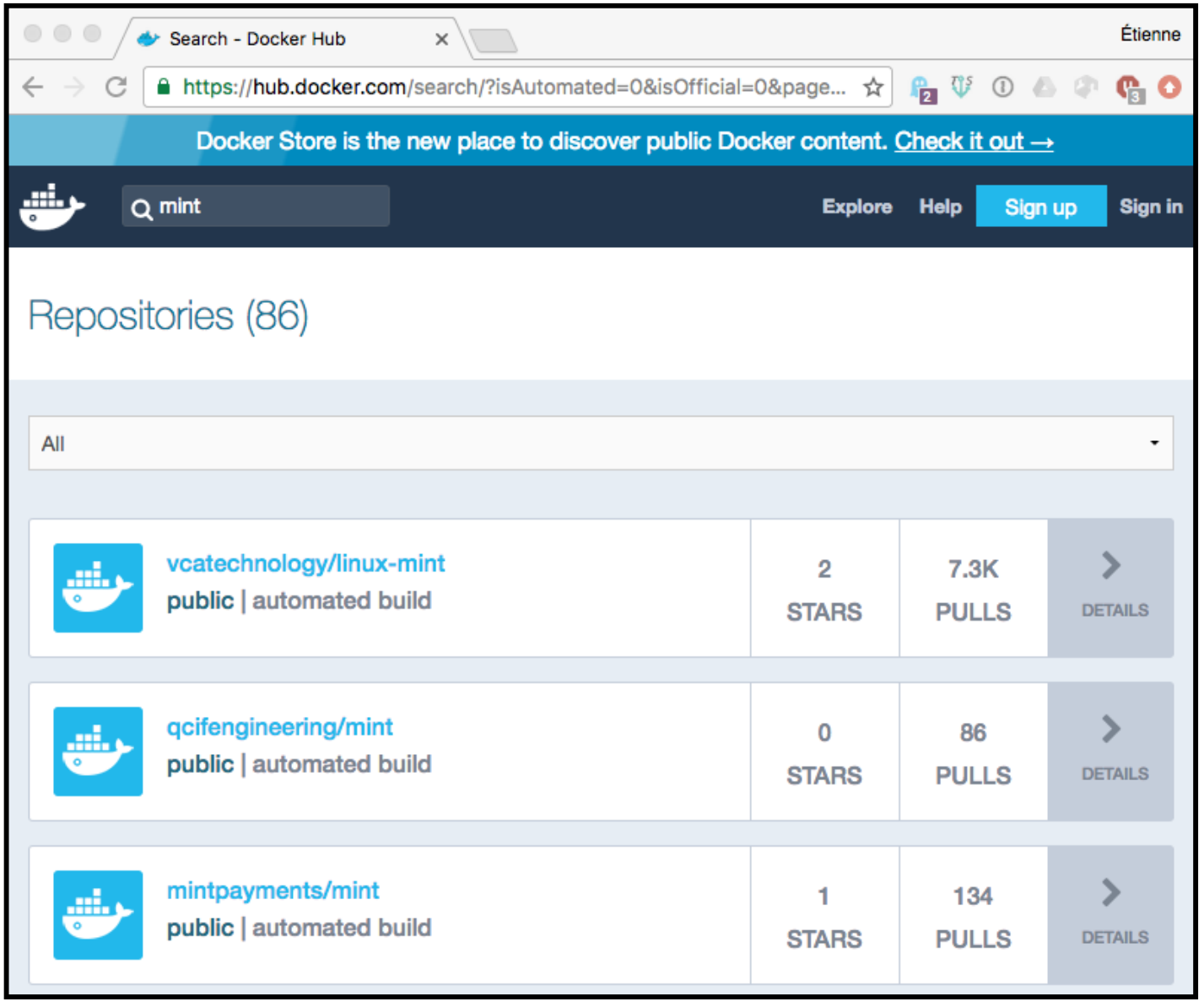
- Made of filesystems layered over each other
  - Principle is called "Union mount"
  - Read-only
    - `bootfs` (Linux boot filesystem)
    - `rootfs` (Base OS file system)
    - applications filesystems
  - New read-write FS on top, when instantiated
- Copy-on-write approach
  - Modified files copied in writeable container
  - Reads go down layers until file found
  - ✓ Identical base read-only file systems shared across containers on same host
- `docker images`
  - List locally-stored images



Dockerhub

- Images are stored in a Docker registry
  - Docker Hub operated by Docker inc.
  - Or your own (company) registry
- Top-level repositories
  - `docker pull ubuntu:latest`
- User repositories
  - `docker pull username/ubuntu:latest`
- Search for images
  - `docker search puppet`

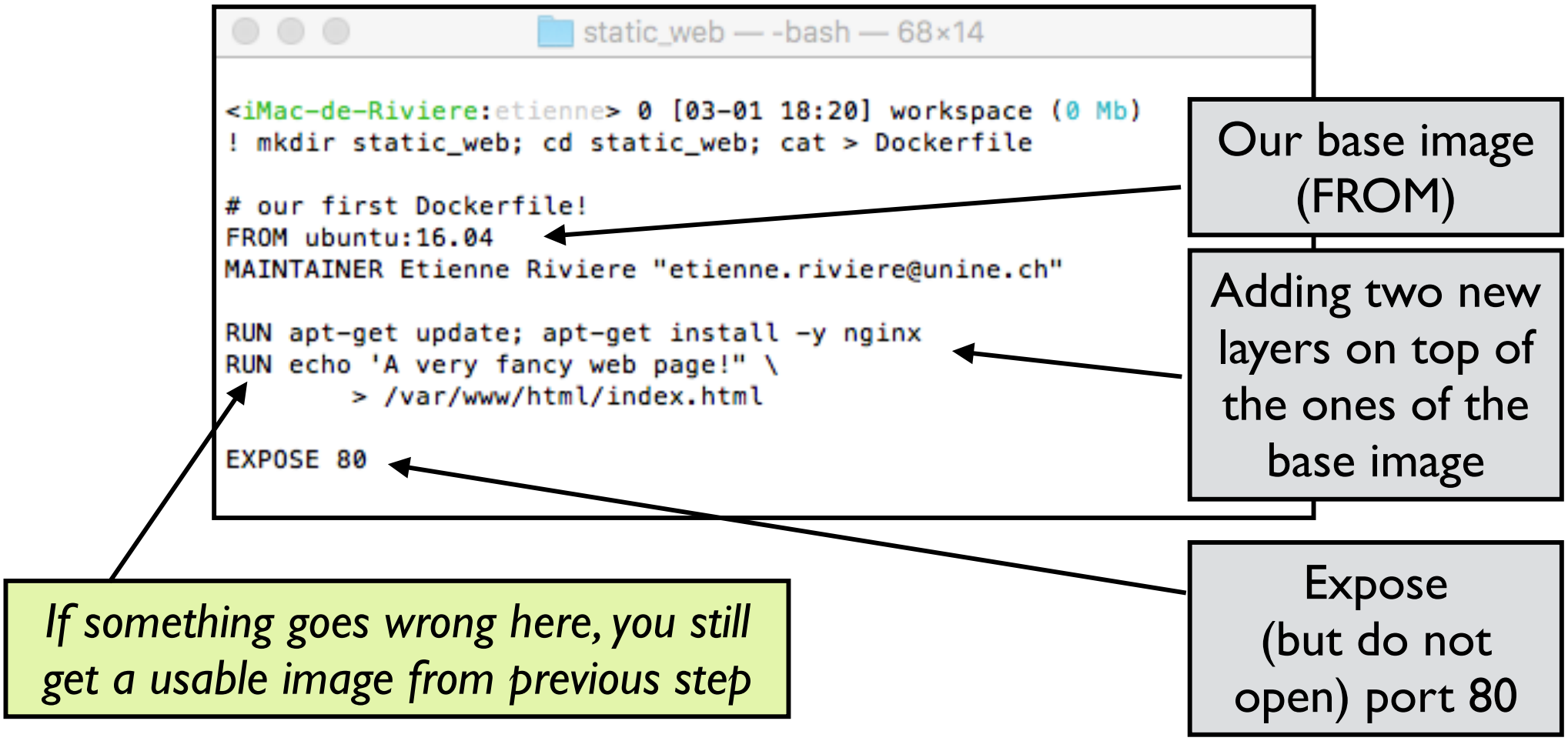
Docker Hub


Search - Docker Hub
Étienne
https://hub.docker.com/search/?isAutomated=0&isOfficial=0&page...
Docker Store is the new place to discover public Docker content. Check it out →
mint
Explore
Help
Sign up
Sign in
Repositories (86)
All
vcatechnology/linux-mint
public | automated build
2
STARS
7.3K
PULLS
DETAILS
qcifengineering/mint
public | automated build
0
STARS
86
PULLS
DETAILS
mintpayments/mint
public | automated build
1
STARS
134
PULLS
DETAILS

Creating images

- An image is typically built from an existing, base image, e.g. `ubuntu:latest`
- Requires writing a `Dockerfile` and running `docker build` in its directory
- Publishing your images to the Docker Hub is possible with a (free) account

Creating a simple Web server



Creating the image

- `docker build` command
- Here, 2nd `RUN` command fails (mismatched ‘ and “)
- Running docker build again starts from the image resulting from 1st `RUN` command
  - Lots of logs from `apt-get install`

```
static_web — -bash — 68×35
<iMac-de-Riviere:etienne> 0 [03-01 18:30] static_web (0 Mb)
[! cat Dockerfile

# our first Dockerfile!
FROM ubuntu:16.04
MAINTAINER Etienne Riviere "etienne.riviere@unine.ch"

RUN apt-get update; apt-get install -y nginx
RUN echo 'A very fancy web page!' \
        > /var/www/html/index.html

EXPOSE 80

<iMac-de-Riviere:etienne> 0 [03-01 18:30] static_web (0 Mb)
[! docker build -t="ccs/static_web" .
Sending build context to Docker daemon 2.048 kB
Step 1/5 : FROM ubuntu:16.04
 ---> 0ef2e08ed3fa
Step 2/5 : MAINTAINER Etienne Riviere "etienne.riviere@unine.ch"
 ---> Using cache
 ---> 30e9f2012e60
Step 3/5 : RUN apt-get update; apt-get install -y nginx
 ---> Using cache
 ---> 2a243823793b
Step 4/5 : RUN echo 'A very fancy web page!'    > /var/www/html/inde
x.html
 ---> Running in 48faaa30c337
 ---> 3ffbfd17b56d
Removing intermediate container 48faaa30c337
Step 5/5 : EXPOSE 80
 ---> Running in 131cbfdd040e
 ---> d1ca87cd1eee
Removing intermediate container 131cbfdd040e
Successfully built d1ca87cd1eee
```

Running the new image

```
static_web — -bash — 64×8
<iMac-de-Riviere:etienne> 0 [03-01 18:30] static_web (0 Mb)
! docker run -d -p 80 --name static_web ccs/static_web nginx -g
"daemon off;"
86b8b0e7bead8565f552d3950d05b7542b491a8beacb851ff1400e5bd46bd851

<iMac-de-Riviere:etienne> 0 [03-01 18:35] static_web (0 Mb)
!
```

- -p option required to open the port
  - for security, exposed ports are not automatically open
- Port 80 has been automatically mapped to local port 32768
  - -p 8080:80 for explicit mapping to 8080 (if available)

```
static_web — -bash — 140×6
! docker ps -l
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS                   NAMES
86b8b0e7bead        ccs/static_web      "nginx -g 'daemon ..."   2 minutes ago       Up 2 minutes        0.0.0.0:32768->80/tcp   static_web

<iMac-de-Riviere:etienne> 0 [03-01 18:37] static_web (0 Mb)
!
```

```
static_web — -bash — 65×5
<iMac-de-Riviere:etienne> 0 [03-01 18:37] static_web (0 Mb)
! curl localhost:32768
A very fancy web page!

<iMac-de-Riviere:etienne> 0 [03-01 18:40] static_web (0 Mb)
```

Some Dockerfile commands

- `CMD`
  - Command to run when launching the container
    - Can be overridden by `docker run` argument
    - If you do not want this, use `ENTRYPOINT` instead
- `WORKDIR / ENV / USER`
  - Set working directory, environment variables, user
- `ADD dir`
  - Copy files from build environment or URL
- `VOLUME`
  - Specific directory that will be outside the union file system and can be shared with other containers, even when not running
  - Allow sharing data between containers
- and many others…

Composing and orchestrating containers

Orchestration and service discovery

- Managing individual containers by hand and "wiring" them together can be a complex process
  - Composition: group several containers together as a single entity
  - Example: web server + database + administration interface (tutorial 2)
- Managing the execution of a fleet of containers is also complex
  - Coordination between multiple Docker hosts
  - Resource management
  - Service discovery
  - Scheduling of the execution of multiple containers
- Some tools building upon Docker for managing sets of containers
  - **Docker compose**: simple container composition
  - **Consul**: service discovery
  - **Docker Swarm**: orchestration and scheduling across multiple hosts
  - **Kubernetes**: large-scale orchestration and scheduling

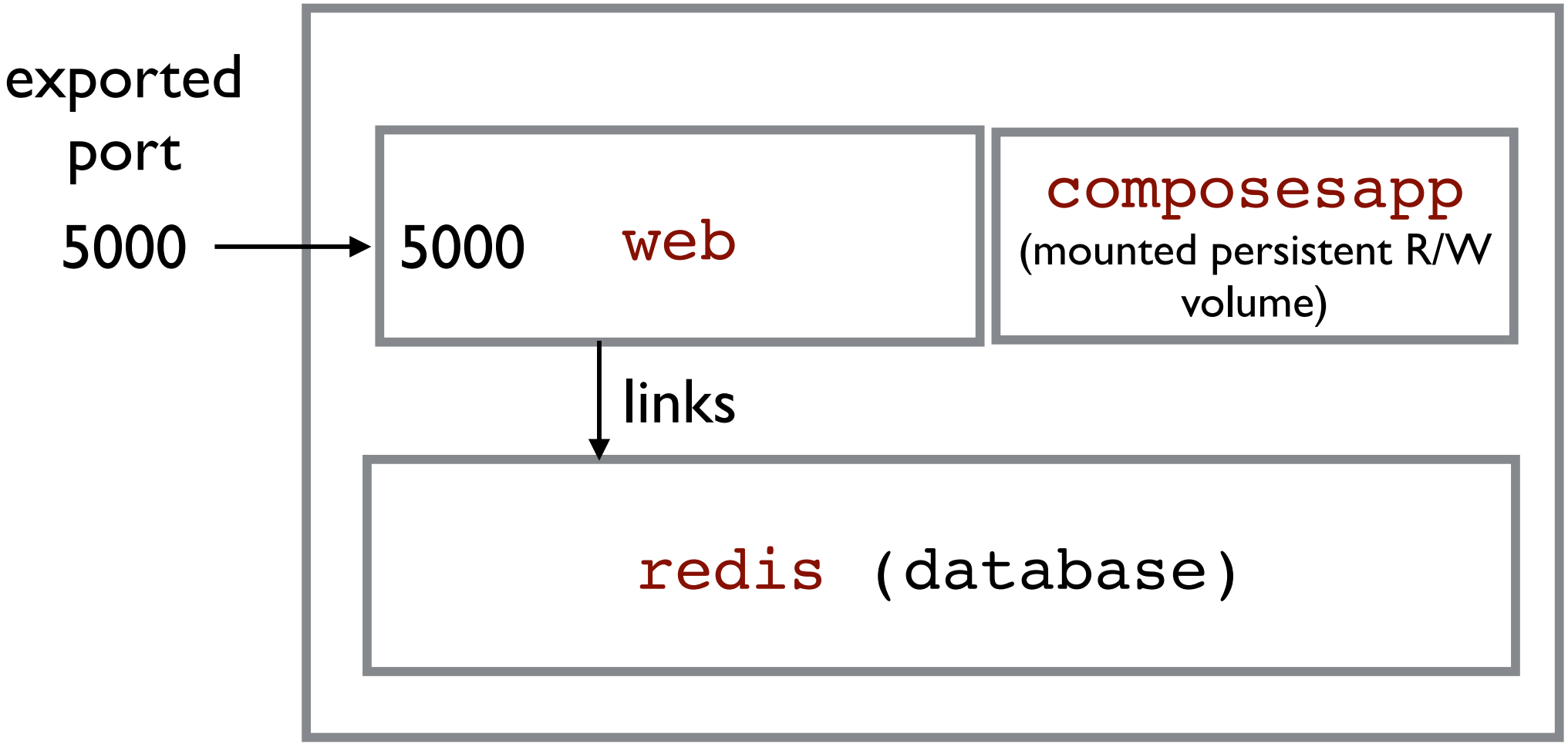
docker-compose

- Define a set of containers (or "services")
  - That must boot up and work together on the same machine
  - Their runtime properties
  - How they are linked
- Described in a YAML file
  - Lists the containers images to instantiate
    - Commands to run
    - Ports to instantiate / export
    - Volumes
  - And how they are linked
    - Container A links container B =
      Ports exported by B available for use from A
    - Service in container B named as "B" from A
    - Automatic setup of Docker network between A and B

An example docker-compose.yml

```
web:
      image: ccs/composeapp
      command: python app.py
      ports:
       - "5000:5000"
      volumes:
       - .:/composeapp
      links:
       - redis
redis:
      image: redis
```

Result


exported
port
5000
5000
web
composesapp
(mounted persistent R/W
volume)
links
redis (database)

Running a docker-compose service

- `docker-compose up`
  - Reads the YAML description `docker-compose.yml`
    - Must be in the directory where the file is
  - Execute both containers
- Outputs (`STDOUT`) prefixed with identifier of the two containers
  - `composeapp_redis_1` and `composeapp_web_1`
  - Run as daemonized containers with `-d` option
- Many docker commands have their compose equivalent
  - `docker-compose ps, stop, start, …`

Consul: service discovery

- Distributed applications formed of many distributed containers providing services
  - Some services are not tightly composed and need to be *discovered* at runtime
- Help from a *service discovery* mechanism
  - Containers register their services to Consul
    - Service name and contact information (`IP:port`)
    - Consul monitors the liveness/availability of services
  - A service catalog
    - Accessible through a DNS-like interface or a HTTP-based REST API for queries

Docker Swarm

- Managing multiple Docker hosts for deploying containers is complex
- Docker Swarm clusters multiple hosts and allows managing them *as a single virtual host*
  - Exposes the regular Docker API over a cluster of hosts
  - Integrated with the regular Docker client
- *Manager* node orchestrating *worker* nodes
  - Dispatches tasks to worker nodes and checks for their liveness
  - Maintains a global view of the cluster and currently running containers
- *Replicated* services: a container (service) is replicated to a number of physical hosts, e.g., for reliability or for horizontal scaling
- *Global* services: a copy of a container (service) runs on each host
- We will cover Docker Swarm in tutorials

Docker and the devops movement

- Traditional separation of roles
  - Dev: build new features
    - Agile development, fast process release
    - Not always sure it deploys well in production
  - Ops: deploy, ensure stability, fix
    - Need to make sure deployed software is consistent and stable
    - May be more conservative in deploying new versions
- DevOps movement: bridge the gap between the two roles
  - Integrated release/test/deployment process
  - Treat everything as code
    - Configuration, hardware descriptions, hardware management (VM)
    - Application deployment (using Docker!)
  - Pairs well with the micro-services approach (next lecture)



- Google has been using Linux containers at a large scale for more than ten years
- A large number of hosts and the need for advanced features led to the development of cluster management systems
  - Dynamic configuration, service discovery, auto-scaling, quota management, security management, etc.
- Three generations of systems
  - Borg (closed source)
  - Omega (closed source)
  - **Kubernetes** (open source since 2014)
    - Tightly integrated with Docker
- The industry standard (Core OS, Red Hat, …) but a complex system to master

Conclusion

- Virtualization: key enabling technology for Cloud computing
  - Enabling the IaaS model and pay-per-use in public and private clouds
- Machine-level (VMs) vs. OS-level (containers)
  - Complementary (not opposing) technologies
- Allow the automatization of infrastructure and application deployment
  - Link with DevOps movement and everything-as-code approach

References

- The Docker book, James Turnbull (e-book)
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, John Wilkes: Borg, Omega, and Kubernetes. Commun. ACM 59(5): 50-57 (2016)
- Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, John Wilkes: Large-scale cluster management at Google with Borg. EuroSys 2015: 18:1-18:17
- Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, John Wilkes: Omega: flexible, scalable schedulers for large compute clusters. EuroSys 2013: 351-364
- Dominik Ernst, David Bermbach, Stefan Tai, Understanding the Container Ecosystem: A Taxonomy of Building Blocks for Container Lifecycle and Cluster Management, technical report, TU Berlin (2015?)
- David Bernstein: Containers and Cloud: From LXC to Docker to Kubernetes. IEEE Cloud Computing 1(3): 81-84 (2014)

